Chapter 15 Review Paper

Mark scheme. Maximum mark: 75

对照哪道真题、这题在卡什么，写在每题下面。原题的图和数字跟这套不一样。

1	12	9618/31/O/N/22 Q10 and 9618/31/M/J/24 Q11
2	12	9618/32/M/J/22 Q5, plus the massively-parallel definition
3	15	9618/32/M/J/22 Q6(b) and 9618/31/M/J/21 Q7
4	14	9618/32/O/N/22 Q8(a)(b)
5	10	9618/32/O/N/22 Q8(c), new expression
6	12	9618/31/O/N/22 Q7 and 9618/31/M/J/23 Q7

Award positive marks. Follow an incorrect truth table or an incorrect but consistent set of loops when the later part says so. Do not award the same fact twice.

Question 1 [12]

(a) [3]

Statement	Answer
An instruction is normally executed in one clock cycle.	RISC
The control unit is microprogrammed.	CISC
The processor has a large number of general-purpose registers.	RISC
Instructions can have different lengths.	CISC
Memory is accessed only by load and store instructions.	RISC
The instruction set provides many addressing modes.	CISC

6 correct rows = 3 marks. 4 or 5 = 2. 2 or 3 = 1. 0 or 1 = 0. A row with both boxes ticked, or neither box ticked, is wrong.

卡点：寄存器多是 RISC。微程序控制单元是 CISC。只有 load 和 store 能访问内存的是 RISC。

(b) [3]

Any three points.

- An instruction is split into stages, for example fetch, decode, operand fetch, execute and write-back. Naming two stages is enough for this point.
- Stages of different instructions overlap, so more than one instruction is in progress in the same clock cycle.
- Each stage is completed in one clock cycle.
- Once the pipeline is full, one instruction can finish on each clock cycle.

- A register between stages holds the result of that stage.

“It is faster” with no mechanism does not score.

(c) [4]

Any four points.

- A RISC instruction takes one clock cycle, so the processor can check for an interrupt after every short instruction.
- A CISC instruction can take several cycles, and the number of cycles varies, so the interrupt waits until that long instruction has finished.
- Several instructions are already in the pipeline when the interrupt is recognised.
- Instructions still in the pipeline are flushed. Accept the alternative: the contents of the pipeline stages are saved in registers and restored after the interrupt.
- The instruction that has reached write-back is allowed to finish. Accept “the current instruction is completed”.
- The program counter and the other registers are saved, the interrupt service routine runs, and they are restored afterwards.
- After the routine, the pipeline has to be filled again, so frequent interrupts remove much of the gain from pipelining.

卡点：只写 “pipeline is flushed” 不够。CISC 那一条指令可能很长，也要写到。

(d) [2]

Any two points.

- Operands are taken from registers, so a typical instruction does not wait for main memory.
- That is what keeps instructions single-cycle, because only load and store instructions access memory.
- Each pipeline stage needs a register for its intermediate value.
- A memory access in the middle of an instruction would stall the pipeline.

Question 2 [12]

(a) [2]

Abbreviation	Full name
SISD	Single Instruction Single Data
SIMD	Single Instruction Multiple Data
MISD	Multiple Instruction Single Data
MIMD	Multiple Instruction Multiple Data

One mark for every two correct rows. Accept Multiple or Many. Accept Instruction or Instructions. Commas are not required. Do not accept a row that swaps Instruction and Data.

(b) [3]

One mark each.

- System 1: SISD
- System 2: SIMD
- System 3: MIMD

Accept the full name. System 3 is not SIMD: the instructions are different, and so are the data.

卡点：System 2 是同一条指令、不同数据。System 3 四个处理器在做不同的指令，是 MIMD。

(c) [3]

Any three points. Repeating only the expanded name does not score.

- There is more than one processor.
- Each processor executes a different instruction.
- Every processor uses the same data, or the same data stream.
- It is rarely used in an ordinary computer because it is difficult to keep several different instructions working on exactly the same data.
- Accept a correct use, such as several processors checking the same input in a fault-tolerant system.

(d) [1]

A computer with a very large number of processors, hundreds or thousands, working at the same time as one system.

“More than one core” does not score. “Many processors working in parallel” does score.

(e) [3]

Any three points.

- The data set, or the task, is divided into smaller parts.
- The processors work on different parts at the same time.
- The processors are connected by a high-speed interconnection network.
- They send intermediate results, or messages, to each other.
- The partial results are combined to give the final result.

Question 3 [15]

P is the output of a 3-input NAND, so P = NOT(A . B . C). Q = B XOR C. R = A OR Q. Z = P AND R.

(a)(i) [4]

A	B	C	P	Q	R	Z
0	0	0	1	0	0	0
0	0	1	1	1	1	1
0	1	0	1	1	1	1
0	1	1	1	0	0	0
1	0	0	1	0	1	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	0	0	1	0

One mark for each of columns P, Q, R and Z, and only if all eight entries in that column are correct. A wrong P does not by itself lose Q or R.

卡点：111 那一行 P 必须是 0。011 那一行 Q 是 0，因为 XOR 的两个输入都是 1。最后一行 R 仍然是 1，但 Z 被 P 拉成 0。

(a)(ii) [2]

Z = NOT A . NOT B . C + NOT A . B . NOT C + A . NOT B . NOT C + A . NOT B . C + A . B . NOT C

Terms may be written in any order. Accept bars or apostrophes. Two marks for all five product terms, each containing A, B and C, and no extra term. One mark for three or four of those terms and no incorrect term.

Follow-through: if column Z is wrong, mark the sum-of-products from the rows where the candidate's Z is 1. Two marks means every such row is present as a full product term and no row with Z = 0 has been included.

A correct simplified expression, such as NOT B . C + A . NOT B + B . NOT C, scores 1 only. The question says not to simplify.

(b) [3]

- Half adder. [1]

- One output is the sum of the two input bits. [1]
- The other output is the carry into the next bit position. [1]

"Sum" and "carry", in either order, are enough for the two purpose marks if the meaning is clear. The name mark still requires "half adder".

(c) [2]

Any two points.

- A multi-bit addition produces a carry from one column.
- That carry has to be added into the next column.
- A half adder has only two inputs, so the carry from the previous column has nowhere to go.
- The circuit with that extra input is a full adder.

(d) [2]

- Sum = $A \oplus B \oplus C_{in}$. Accept A XOR B XOR C_{in}. [1]
- $C_{out} = (A \,.\, B) + (A \,.\, C_{in}) + (B \,.\, C_{in})$. [1]

Accept $C_{out} = (A \,.\, B) + (C_{in} \,.\, (A \oplus B))$. Do not accept Sum = A XOR B. That is the half adder.

(e) [2]

- OR gate. [1]
- Its inputs are the carry outputs of the two half adders. [1]

"The two carry signals" is enough for the second mark.

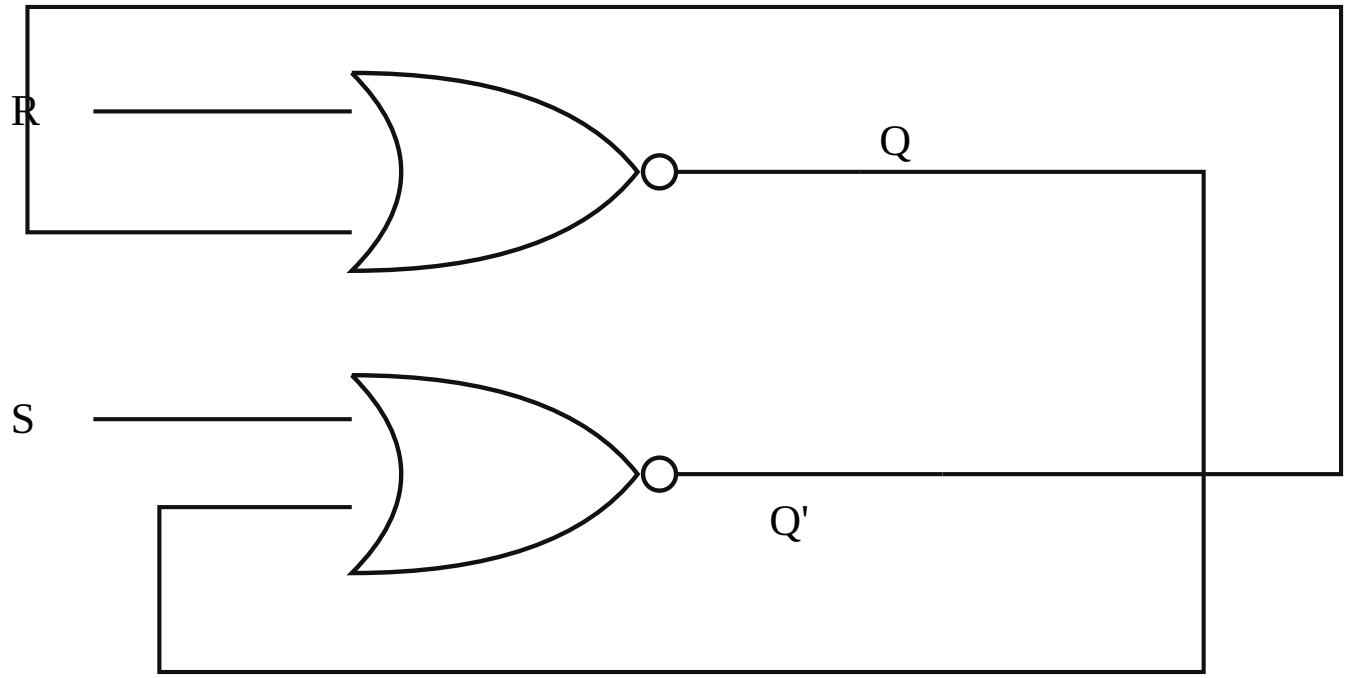
Question 4 [14]

(a) [2]

- Combinational: the output depends only on the current inputs. [1]
- Sequential: the output depends on the current inputs and on a previous output, or on a stored state. [1]

Accept "combination circuit", which is the wording used in the Hodder chapter.

(b) [3]



S goes into the NOR gate whose output is Q'. R goes into the NOR gate whose output is Q. Q is fed back into the lower gate, and Q' is fed back into the upper gate.

- Two NOR gates, each with an output bubble or an otherwise clear NOR symbol. [1]
- Cross-coupling: Q is an input of the other gate, and Q' is an input of the first gate. [1]
- Labels are placed so that S = 1 sets Q to 1, and R = 1 resets Q to 0. Q' is the second output. [1]

A mirror image scores the third mark if S still sets and R still resets. If S and R are swapped relative to the outputs, the third mark is lost and the other two can stand. A NAND circuit does not score the first mark. It can score the cross-coupling mark.

卡点：S 要接到输出 Q' 的那个 NOR。接到输出 Q 的那个门上，S = 1 会把 Q 清成 0。

(c) [4]

S	R	Q	Q'
0	0	no change	no change
0	1	0	1
1	0	1	0
1	1	invalid	invalid

One mark for each row. For 00 accept hold, memory, latch, or “Q is unchanged”. A fixed 0 or 1 on that row scores 0. For 11 accept invalid, not allowed, race, unpredictable, or Q = 0 and Q' = 0. Writing Q = 1 on that row scores 0.

(d) [2]

Any two points.

- Both NOR gates receive a 1, so both outputs are forced to 0.
- Q' is then not the opposite of Q.
- When S and R later return to 0, the latch can settle in either state. The next output is not predictable.

(e) [1]

It stores one bit, or it is a one-bit memory. Accept “flip-flops are used to build a register” or “it retains the previous output”.

“It is a sequential circuit”, with no idea of storage, does not score.

(f) [2]

- Q changes to the opposite of its current value. It toggles. [1]
- J = 1 and K = 1 has a defined result, so the invalid combination from the SR flip-flop does not arise. [1]

The toggle mark needs the change of state. “Q = 1” does not score, because the result depends on the current Q.

Question 5 [10]

(a) [2]

- NOT(A . B) = NOT A + NOT B. [1]
- NOT(A + B) = NOT A . NOT B. [1]

Any letters are acceptable. The operator must change. A law that keeps AND as AND, or OR as OR, scores 0 for that law.

(b) [6]

X = NOT(A . NOT(B + C)) . (B + A . NOT B)

1. NOT(B + C) = NOT B . NOT C. De Morgan. The expression is now NOT(A . NOT B . NOT C) . (B + A . NOT B).
2. NOT(A . NOT B . NOT C) = NOT A + NOT(NOT B) + NOT(NOT C). De Morgan. The operator inside the bracket changes from AND to OR.
3. NOT(NOT B) = B and NOT(NOT C) = C. Double complement. The left bracket is now NOT A + B + C.
4. B + A . NOT B = (B + A) . (B + NOT B). Distributive law, X + Y.Z = (X + Y)(X + Z).
5. B + NOT B = 1, and Y . 1 = Y. Complement and identity. The right bracket is now A + B.
6. Expand (NOT A + B + C) . (A + B). NOT A . A = 0 and B . B = B, so this is NOT A . B + A . B + B + A . C + B . C.
7. Absorption: B + NOT A . B = B, B + A . B = B and B + B . C = B. Therefore X = A . C + B.

Final answer: **X = A . C + B**. Accept B + A . C, or the same expression with bars.

Award any six of the points above, up to 6. The correct final answer is one of those points. A correct final answer with no working scores 1. An incorrect final answer can still score working points that were completed before the error.

A shorter expansion is acceptable for the last two points if it reaches A.C + B and the laws are named. Stopping at NOT A + B + C has not used the second bracket and is not the final answer.

卡点：先把 NOT(B + C) 拆开，再拆外面那个 NOT。B + A.NOT B 不是 B。它等于 A + B。最后两项吸收完只剩 A.C + B。

(c) [2]

- Double complement and De Morgan: A.C + B = NOT(NOT(A.C + B)) = NOT(NOT(A.C) . NOT B). [1]
- NOT(A.C) is A NAND C, and NOT B is B NAND B. So X = (A NAND C) NAND (B NAND B). [1]

Accept NAND(NAND(A, C), NAND(B, B)). The correct NAND expression on its own scores the second mark. NAND(NAND(A, C), B) is wrong: that is A.C + NOT B.

Question 6 [12]

(a) [1]

An expression made of AND terms, or product terms, joined by OR.

“A Boolean expression” on its own does not score. Accept “minterms joined by OR”.

(b) [1]

Any one of:

- Groups of 1s that can be combined are visible, so a simplification is less likely to be missed than in algebra.
- The minimal sum-of-products can be read from the groups.
- Cells in a group differ by the variables that are eliminated, so each term is read from the headings that stay the same.

“It is faster” or “it is easier”, with no reason, does not score.

(c) [2]

- The headings are a Gray code. Each heading differs from the next by one bit. [1]
- Adjacent cells, including the pair of cells that wrap around from the first column to the last column, then differ by only one variable. A group can eliminate the variable that changes. [1]

The order 00, 01, 10, 11 makes the first and last columns differ by two bits, so those cells are not a legal pair.

(d) [2]

CD \ AB	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	1	1	1
10	0	1	1	1

wrap pair B.C A.C in both quads

Two marks if all 16 cells are correct. One mark if exactly one cell is wrong. Zero if two or more cells are wrong. The colours are for the loops in part (e). A cell with no 0 and no 1 is wrong.

卡点：ABCD = 1000 在表里是第 9 行，放进图里是第一行、最后一列，不是第三列。二进制顺序和 Gray code 不是同一列。

(e) [3]

One mark for each loop:

- The quad in columns AB = 11 and AB = 10, rows CD = 11 and CD = 10. This is A.C.
- The quad in columns AB = 01 and AB = 11, rows CD = 11 and CD = 10. This is B.C.
- The pair in row CD = 00, columns AB = 00 and AB = 10. This pair wraps around the ends of the row. It is NOT B . NOT C . NOT D.

An incorrect extra loop loses one mark. The minimum for this part is 0. A loop around all six 1s in the bottom-right of the map is incorrect: a group of six is not allowed, and columns 01 and 10 are not adjacent. Two separate single-cell loops in place of the wrapping pair score 0 for that loop.

(f) [2]

X = A.C + B.C + NOT B . NOT C . NOT D

- A.C + B.C, with no third term taken from those two quads. [1]
- The wrap term NOT B . NOT C . NOT D is present, and the whole expression has no extra term. [1]

Terms may be written in any order. Follow-through from a wrong but legal set of loops: award the expression that those loops actually produce, using the same two-mark split.

(g) [1]

X = C . (A + B) + NOT B . NOT C . NOT D

Accept (A + B) . C for the first part. Also accept C . (A + B) + NOT(B + C + D), because NOT B . NOT C . NOT D = NOT(B + C + D). The unfactorised answer from part (f) does not score again. Dropping the third term scores 0.

卡点：A.C + B.C 要提出 C。绕边那一项不能丢掉，它和这两个四格圈没有重叠。

The circuit table, the adder identities, the algebra and the K-map were checked by evaluating every input combination in `verify_logic.py`.